ENGINEERING A DISTRIBUTED OBSERVABILITY PLATFORM

From Vendor-Driven Monitoring to Open-Source Observability

Cloud-Native Collection

Distributed telemetry gathering powered by open standards and scalable pollers.

Global Metrics

Unified multi-region querying and long-term storage across infrastructure.

Actionable Incidents

Automated event correlation and context-rich alerts for rapid response.

Uber Engineering | SREday Amsterdam

Giovanni Pepe
Razvan Cicu

The challenge: monitoring is not observability

Fragmentation

Heterogeneous network vendors and telemetry interfaces

Limited Flexibility

Disconnected monitoring, alerts and incident workflows

Lack of Operational Context

Raw signals without shared site or device context

Scale & Automation Bottleneck

Operational scale demands automation, not more dashboards

Architecture principles

Vendor Agnostic

Standardized telemetry interfaces across multi-vendor hardware.

Open Source Core

Built on open-source tools like Prometheus, Telegraf, Thanos & Alertmanager.

Regional Autonomy

Local collection and alert evaluation close to infrastructure.

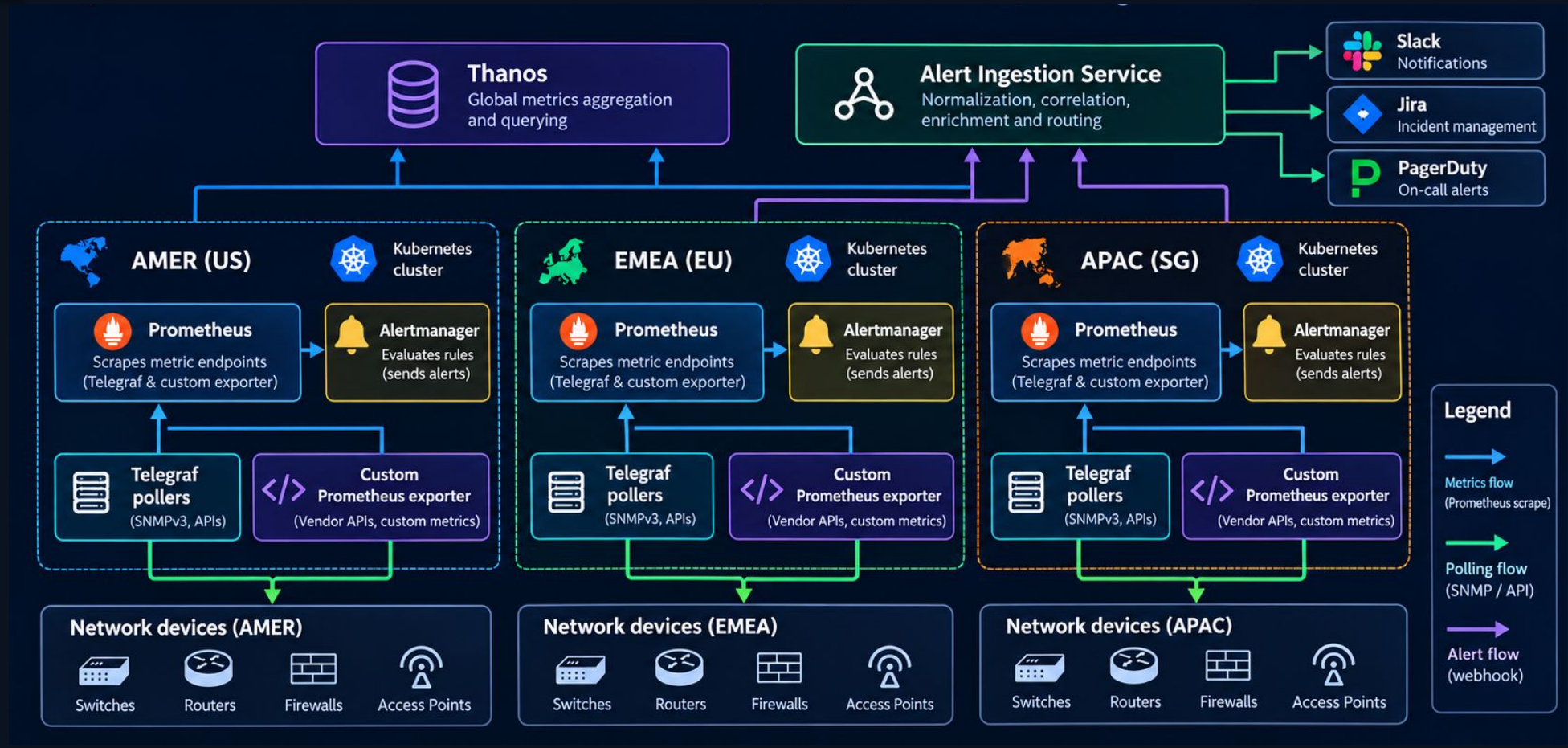
Cloud-Native Scalability

Kubernetes-based deployment; scale pollers & processors independently as well as regional distribution.

Extensible Pipeline

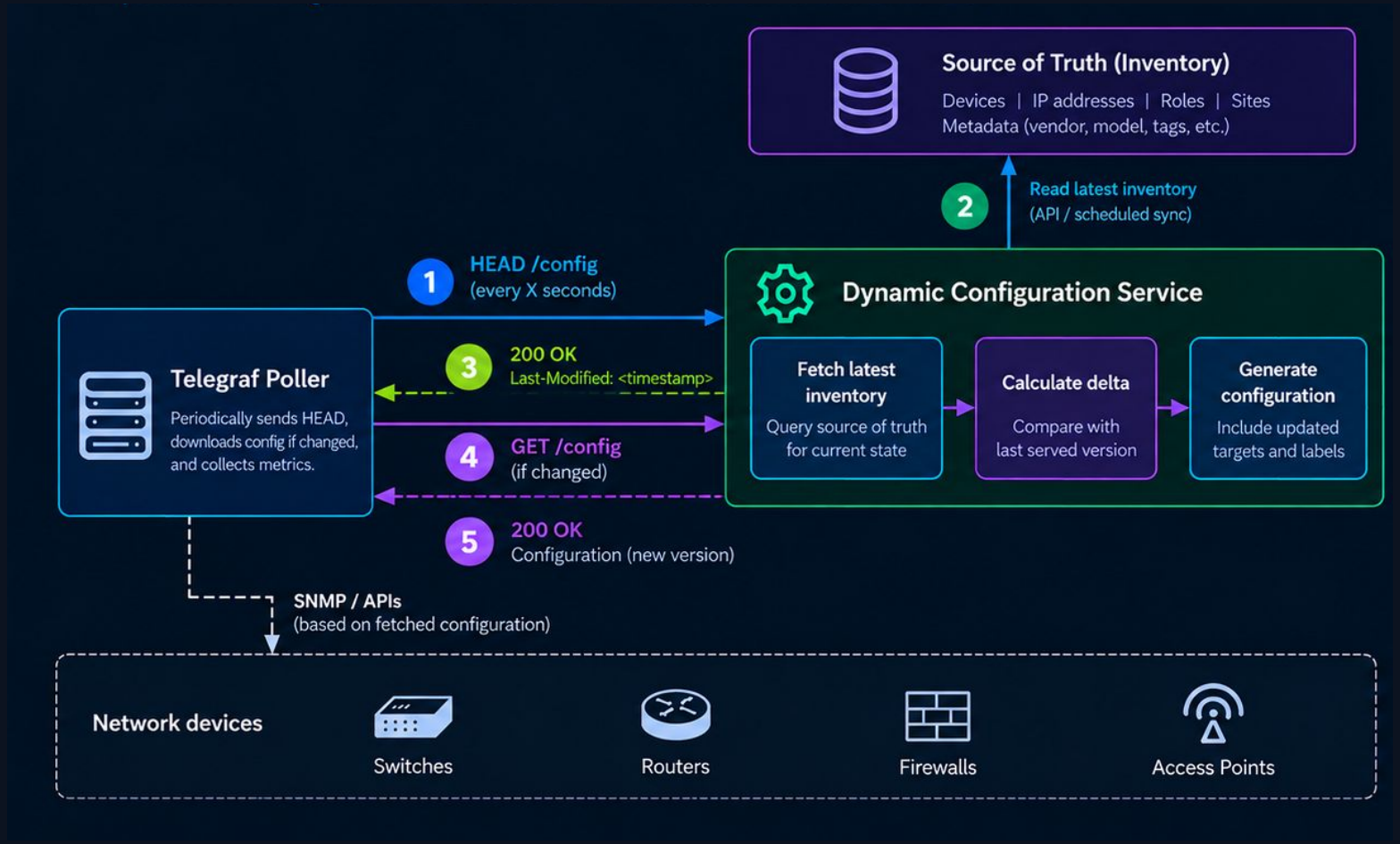
Integrate new vendors and telemetry sources seamlessly into core workflows.

Distributed Observability Architecture



Dynamic Configuration: Keeping Monitoring in Sync With the Network

A monitoring system is only as accurate as the source of truth driving its configuration



1 **Telegraf sends HEAD**
Each Telegraf poller sends an HTTP HEAD request for its configuration at a configured interval, including its last known timestamp (If-Modified-Since).

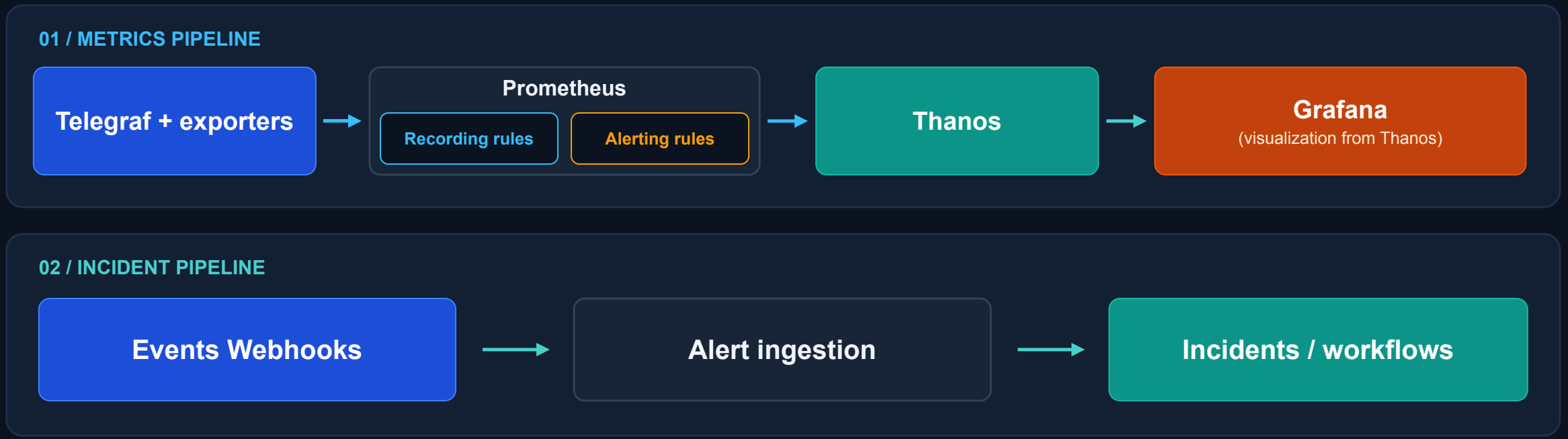
2 **Dynamic config reads inventory**
The service queries the source of truth for the latest device inventory and metadata.

3 **Compute delta and respond**
The service compares the latest inventory with the last served version. It responds to the HEAD with 200 OK and a Last-Modified header (updated only if there are changes).

4 **Telegraf sends GET (if changed)**
If the Last-Modified value is newer than the one stored locally, Telegraf requests the full configuration with a GET.

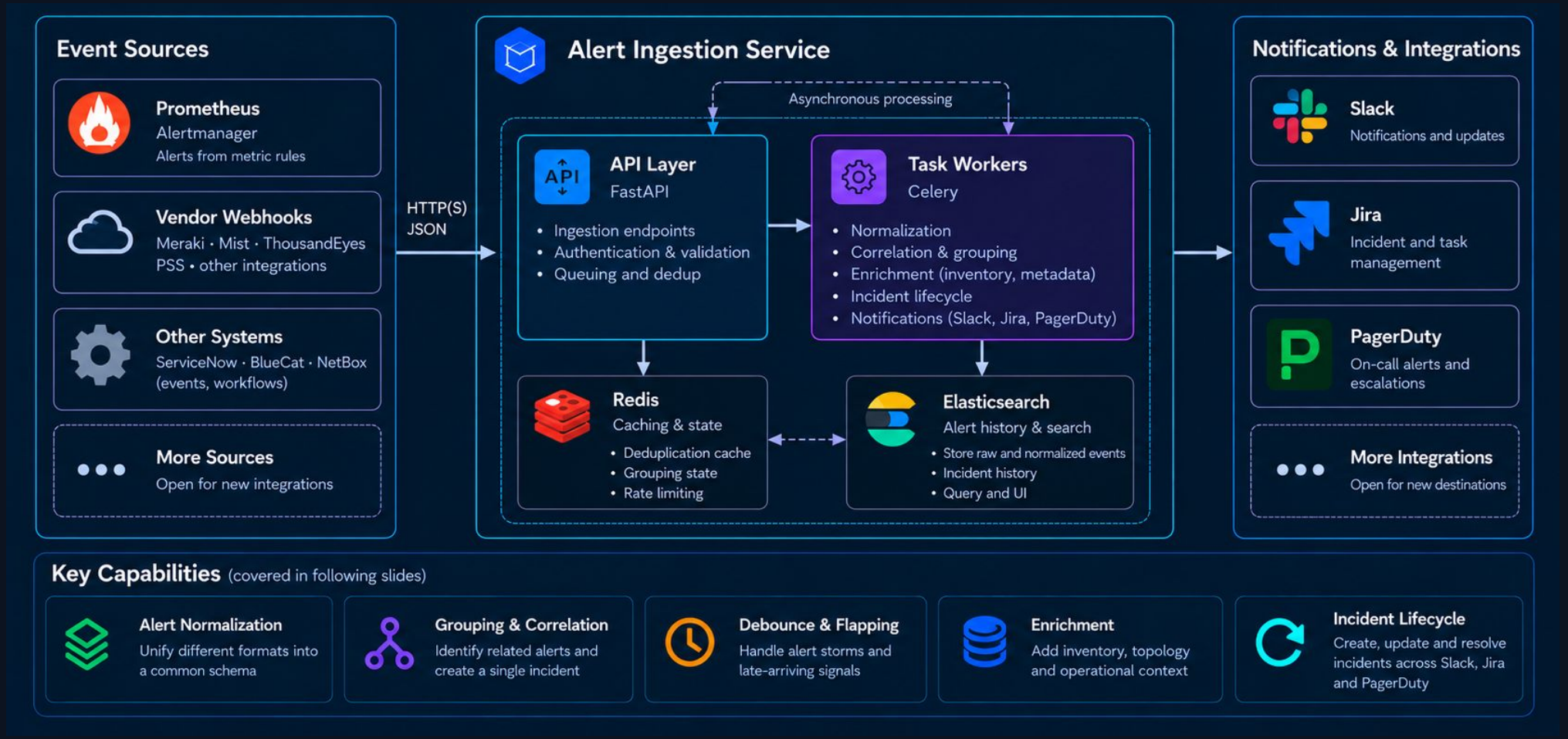
5 **Serve updated configuration**
The service returns the new configuration, and Telegraf updates its local configuration and starts (or adjusts) monitoring the targets.

Two separate paths: metrics and incidents



Alertmanager feeds the incident path; metrics are queried through Thanos.

Alerts Ingestion Service



Not every signal should become an incident

METRIC-BASED ALERTS

High coverage
Potentially noisy
Supporting evidence

VENDOR INSIGHTS

Higher-level semantics
Context-aware signals
Potential incident triggers

The challenge: Volume + Signal Quality + Context = Actionable Incidents

Alert lifecycle: a stateful system

BUFFER

Debounce & Collect

- Creates temporary Redis group on first signal arrival
- Wait window (~2.5 min or configurable) aggregates related alerts
- Enforces max wait time to avoid notification delays

→

FLUSH

Atomic Claim & Create

- Atomic Redis Lua scripts prevent worker race conditions
- Single Celery worker claims group for execution
- Generates parent Jira ticket & main Slack alert

→

OPEN

Enrich & Maintain Context

- Group stays open (~10 min or configurable) to receive late events
- Appends new signals directly to existing Slack thread
- Dampens flapping alerts without creating extra tickets

→

RESOLVE

Close & Archive

- Transitions state upon condition resolution
- Preserves entire operational timeline in Jira
- Cleans up state machine records in Redis

Operational context and AI summaries

AI AGENT ANALYSIS

- Analyzes correlated events, metadata, and site context
- Generates concise summaries of complex failures
- Connects related signals across switches, APs, and WAN

ACCELERATED TRIAGE

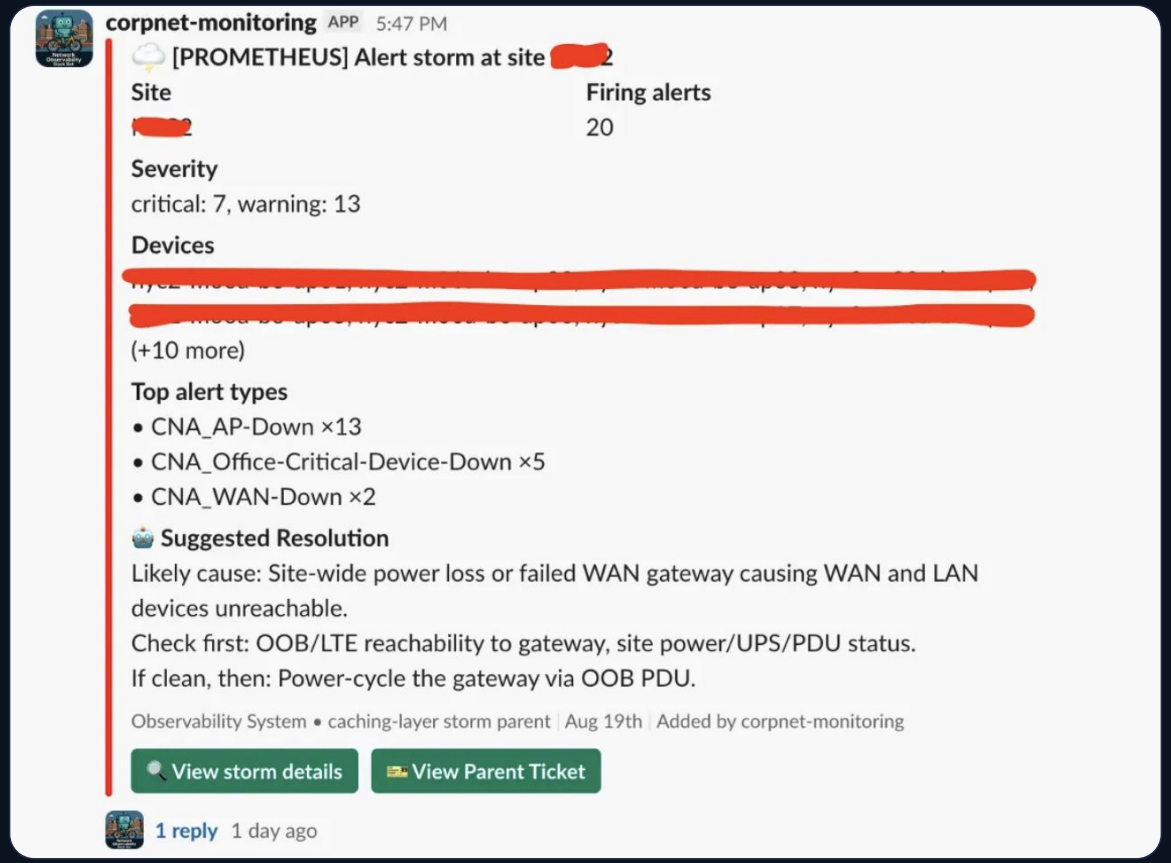
- Helps engineers understand root causes faster
- Preserves all raw signals and evidence for deep dives
- Speeds up initial investigation without replacing humans

Integrated Workflow: Slack, Jira and PagerDuty become incident views, not alert dumps

Case study: one office outage

Multiple raw alerts from affected site → **Group by site and incident context** → **One actionable incident**

- Primary symptoms: loss of connectivity across both ISP links
- Supporting signals become evidence, not separate tickets
- Slack thread and Jira issue retain the operational timeline



Production lessons and takeaways

01

Distribute collection; unify visibility

02

Separate metrics from incident semantics

03

Design correlation for concurrency and failure

Questions?

Thank you for attending — open for Q&A and discussion.